RESEARCH IMPLEMENTATION / EVIDENCE RECORD

Diabetic retinopathy grading with Vision Mamba

Architecture, data controls, verification and delivery status

Dharan Tej Reddy 4 November 2026

Scope of this document. This implementation dossier records the code and preparation work. It is **not** the assignment's final experimental report. A full training run, held-out classification metrics, prediction figures and a result-bearing LaTeX report do not yet exist. Two recorded optimizer steps are qualification checks only.

Project links

GitHub source repository · Vision Mamba paper · Original EyePACS release

Source code and documentation are public. Retinal images, labels, credentials, private access links and diagnostic artifacts remain in private storage.

1 Assignment and delivery state

The internship brief asks for a Vision Mamba classifier implemented from scratch in PyTorch, a custom EyePACS `Dataset` used with `DataLoader`, five-grade training, held-out EyePACS and Messidor evaluation, metrics and example predictions, and a compiled LaTeX results report. It evaluates code quality, architecture understanding, analysis, documentation and reproducibility. The last two output groups still depend on a completed experiment.

Expected deliverable	Existing implementation	Evidence state
PyTorch Vim	Own model blocks, reference and parallel scan, five-logit head	Source exists; earlier revision passed two real-image optimizer steps
Dataset/preprocessing	Own retinal `Dataset`; full-field transform; manifest, split and audit; PyTorch `DataLoader`	35,126 original EyePACS images audited privately
Train/evaluate scripts	CLI, qualification, checkpointed training, sealed evaluation, metrics and report generator	Code tested; full epoch and sealed evaluation unrun
Metrics and predictions	Result generators reject incomplete runs	No measured test results
Compiled results report	LaTeX generator and compiler smoke check	No final experiment PDF

For a code-focused review, the first three deliverables are available as source. That is a reduced-scope submission against the PDF, because the PDF also asks for real training and results.

2 Architecture and paper mapping

The Vision Mamba paper represents images as sequences of position-marked patches and uses selective state-space processing in both directions to build global context [1]. This implementation uses a middle class token and independent forward/reverse convolution and SSM parameters. It does not import a ready-made Vim model or pretrained weights. The authors' implementation informed the architecture review, not the model dependency.

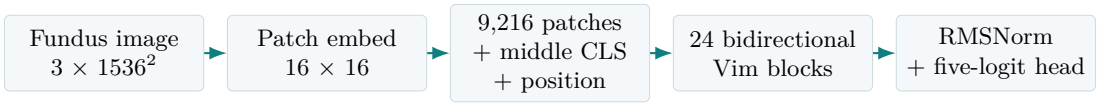


Figure 1. Implemented inference path. These are configured dimensions, not a measured clinical operating point.

For image side $H = 1536$, patch side $P = 16$, width $D = 768$ and five grades, the token count and shapes are

$$N = (H/P)^2 = 96^2 = 9{,}216, \qquad L = N + 1 = 9{,}217,$$

$$\text{patch tokens} \in \mathbb{R}^{B\times N\times 768}, \qquad \text{class logits} \in \mathbb{R}^{B\times 5}.$$

The configured block uses expansion factor two, directional width $E = 1{,}536$, state dimension $S = 16$, causal convolution width four, and depth 24. The instantiated classifier has 103,760,645 trainable parameters. The 1536-pixel input and 300-epoch schedule are project choices, not settings mandated by the paper or assignment brief.

3 Bidirectional selective state-space block

Every block RMS-normalizes its input, projects features and a shared gate, and processes the features forward and in reverse token order. Each direction has its own causal depthwise convolution, input-dependent state projections, step-size projection, negative diagonal state matrix and skip coefficient. The outputs are returned to the original order, summed, gated, projected and added to an FP32 residual.

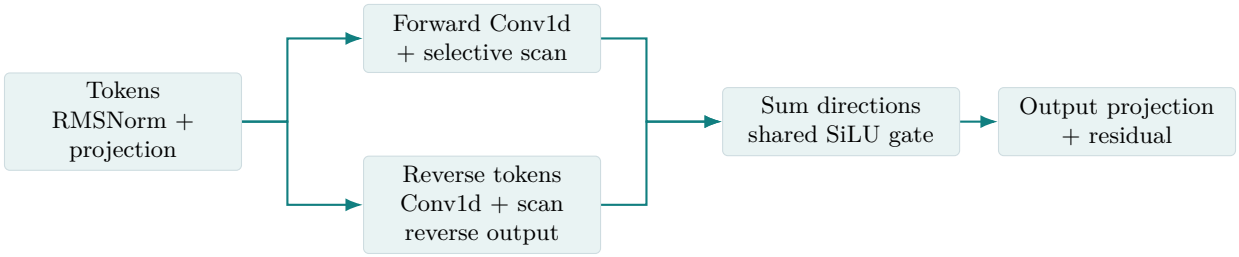


Figure 2. The forward and reverse SSM/convolution parameters are independent; the gate is shared.

In the reference scan, Δ_t and B_t, C_t depend on current input, $A = -\exp(A_{\log})$, and the state update uses FP32:

$$\bar{A}_t = \exp(\Delta_t A), \qquad h_t = \bar{A}_t \odot h_{t-1} + \Delta_t B_t \odot x_t, \qquad y_t = \langle C_t, h_t \rangle + D_{\text{skip}} \odot x_t.$$

The input drive follows the released Mamba/Vim selective-scan convention $\Delta_t B_t$ [2]. The primary parallel backend composes affine transitions in bounded chunks and implements its own backward recurrence. It does not reproduce the paper's fused SRAM kernel, so the paper's published speedups are not claims about this implementation.

Path	Purpose	Present evidence
Reference	Transparent recurrence and gradient comparison	Mathematical source; local parity tests
Parallel	Primary from-scratch PyTorch training path	Earlier BF16 source passed real-image parity and two model steps
Pinned CUDA	Optional low-level scan efficiency path, never a ready-made Vim model	Scan-only benchmark passed; current whole model unqualified

Implementation map

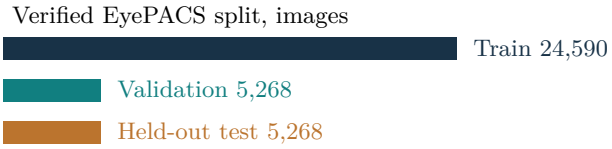
`src/drvim/model/vim.py` defines the classifier, blocks and directional mixers. `src/drvim/model/scan.py` defines the reference, in-house parallel and optional pinned kernel paths. `src/drvim/qualify_scan.py` compares numerics and runs real-image optimizer preflight before training can accept a qualification artifact.

4 Data architecture and leakage controls

The original EyePACS training release has 35,126 labeled JPEGs. The private preparation job read all five official archive parts, checked archive CRCs, decoded and SHA-256 hashed every image, reconciled it to a label, and assigned a grade-stratified split that keeps filename-paired left and right eyes together.



Figure 3. Preparation and test boundary. Filename pairing tokens do not independently prove unique patient identity.



The active custom `RetinaDataset` loads records for its declared source and split. Its transform applies EXIF orientation, converts to RGB, rescales without cropping the retinal field, pads to a square, and allows horizontal flipping only for training. PyTorch's `DataLoader` supplies workers and batching; no ready-made retinal dataset loader is used.

A separate high-assurance candidate adds pinned manifest identity, per-read byte hashing, size and format limits, ICC-to-sRGB conversion, deterministic augmentation, and metadata evidence gates. It is **not connected to the primary training run**. The public EyePACS release lacks verified patient linkage, site, camera, acquisition date, adjudication and rights metadata; the candidate mode therefore fails closed instead of inventing those fields.

Concern	Active assignment loader	Optional high-assurance candidate
Source identity	Manifest checked before run	Manifest pinned and image bytes rehashed at sample read
Color	RGB conversion	Embedded ICC to sRGB; untagged assumption recorded
Geometry	Full-field letterbox	Same geometry plus occupancy metadata
Augmentation	Train-only random flip	Seed/epoch/image-ID deterministic flip
Clinical metadata	Unavailable	Explicit patient/site/device/grade/rights attestations required

Source map for data handling

File	Responsibility
`data/prepare.py`, `data/manifest.py`	Official cohort adapters, labels, hashes and strict manifest schema
`data/split.py`, `data/audit.py`	Grouped assignment and leakage/source checks
`data/dataset.py`, `data/preprocess.py`	Active custom Dataset and full-field image transform
`data/high_assurance_dataset.py`	Separate candidate for stronger provenance and image controls

Official original Messidor preparation

All 12 archives and 12 canonical annotation workbooks passed private integrity checks. The preparer decoded and SHA-256 hashed 1,200 TIFFs, applied four published grade corrections, and excluded 13 published Base33 duplicates. Its schema-valid manifest has 1,187 eligible external images: R0 547, R1 149, R2 240 and R3 251. Included image SHA-256 sets have zero exact overlap with the verified EyePACS manifest; near duplicates are not excluded by this check. No external model predictions exist.

5 Training and external evaluation

The classifier is randomly initialized. The training code defines cross-entropy, AdamW with parameter-specific decay exclusions, warmup and cosine learning rate, mixed precision, gradient clipping, accumulation, activation checkpointing and distributed data parallel execution. Each completed epoch saves model, optimizer, scheduler, scaler, RNG, config and manifest identity. Highest EyePACS validation QWK selects the checkpoint. The held-out EyePACS and Messidor tests never select hyperparameters.

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{M}\sum_{i=1}^{M} \log p_\theta(y_i \mid x_i), \qquad y_i \in \{0,1,2,3,4\}.$$

Distributed padding examples have zero contribution so each real training image contributes once to the global loss. An epoch-boundary pause saves a checkpoint without writing the full-run completion marker.

Declared configuration (not a completed run)

Setting	Value	Setting	Value
Epochs / seed	300 / 2026	Batch / accum.	2 / 4 per GPU
AdamW learning rate	10^{-4}	Decay / warmup	0.05 / 5 epochs
Precision	FP16 AMP	Clipping / drop path	1.0 / 0.1
Workers per rank	8	Selection criterion	Validation QWK

Original Messidor is the external dataset named by the brief. Its R0–R3 grades differ from EyePACS ICDR 0–4; five-class QWK against original Messidor would be invalid. The declared cross-source binary proxy is Messidor R2/R3 versus model ICDR 2/3/4. Optional Messidor-2 uses separately published adjudicated grades 0–4 [4, 5]. Neither external cohort enters training or checkpoint selection. ADCIS states that 1,058 Messidor-2 PNGs come from original Messidor and 690 JPGs form an extension; the two named cohorts are therefore not independent replications.

The evaluation command requires a completed-run marker, pins the source manifest and selected checkpoint, verifies images, then records per-image probabilities and predictions. It calculates accuracy, precision, recall, F1, QWK where valid, confusion matrices and loss curves. The report generator recomputes metrics from saved rows before placing plots and image pairs in LaTeX. **These generators exist; no classification results have been generated yet.**

Declared metric definitions

For a five-grade cohort, the confusion matrix contains integer counts $C_{ij} = \#\{n : y_n = i, \hat{y}_n = j\}$. Accuracy is the fraction of correct predictions; class precision, recall and F1 are computed with explicit support. Macro values average the five class values equally. Quadratic weighted kappa is

$$\kappa_{\mathrm{QW}} = 1 - \frac{\sum_{i,j} w_{ij} O_{ij}}{\sum_{i,j} w_{ij} E_{ij}}, \qquad w_{ij} = \frac{(i-j)^2}{(K-1)^2}, \quad K = 5,$$

where O is observed grade-pair frequency and E is the expected frequency from the observed marginals. Undefined QWK is rejected rather than replaced with a fabricated zero. Original Messidor receives its declared binary endpoint only; five-grade QWK is reserved for commensurate ICDR labels.

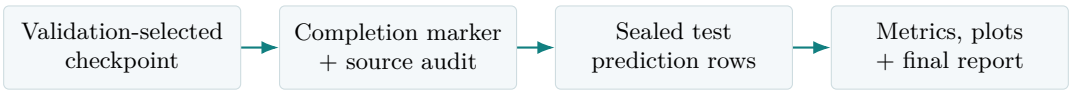


Figure 5. Implemented result gate. The rightmost artifacts cannot exist honestly until the preceding experimental stages complete.

6 What has actually run

Check	Observed result	Interpretation
Original EyePACS preparation	35,126 labeled images decoded and hashed; all five grades in each split; no filename-pair group crosses splits	Complete data audit, not model training
Earlier BF16/parallel qualification	One Tesla T4, 1536-pixel input, 103,760,645 parameters; two eight-image optimizer steps of 195.71 and 197.24 seconds; peak allocation 4.64 and 5.47 GB	Two-step diagnostic only
Scan-only comparison	At 9,217 tokens on T4, in-house FP32 scan 0.334 s versus pinned CUDA scan 0.0256 s; forward maximum absolute difference 1.31×10^{-6}	About 13 times faster for this scan benchmark only
Repository checks	Ruff passed and 12 local tests passed	Source checks, not accuracy

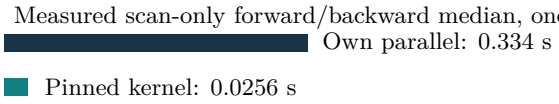


Figure 4. This compares only a selective-scan direction, not a complete model step, epoch or classifier.

The kernel timing does not establish full-model throughput. The two-step qualification is pinned to an earlier BF16/parallel source revision. The current FP16/optional CUDA code requires its own exact-source qualification. At the earlier T4 rate, the configured 300-epoch experiment cannot fit the available free Kaggle time budget. The brief does not require 300 epochs. No paid environment has been used.

7 Stack and reproducibility

Technology	Role and boundary
Python 3.10+	Package, CLI, manifests and acquisition verification
PyTorch	Tensors, autodiff, `Dataset`/`DataLoader`, AMP, DDP and layers; model and training loop are authored in this repository
NumPy / pandas	Array conversion, annotation tables and manifests
Pillow	JPEG/TIFF decode, EXIF correction, resize and optional color control
scikit-learn	Metrics and confusion counts, not a ready-made retinal model
Matplotlib	Plots only after actual evaluation
PyYAML	Recorded experiment configuration
xlrd / openpyxl	Official Messidor workbook parsing
Tectonic / pdfLaTeX	Result report compilation
Kaggle	Private EyePACS input and GPU diagnostic jobs
Git / GitHub	Public source and docs; no images, credentials or weights

The public repository is github.com/dharan1007/internship-task-1. The audited EyePACS split manifest SHA-256 is `f933f1bcff8f99688fd8a44984acaa4ce0c929d899be9bddcabdfc54ea7c2117`. The hash identifies the exact split; raw licensed images remain private. Source, manifest, config, checkpoint and report artifact identities are recorded so a finished experiment can be traced back to its inputs.

8 Access and support timeline

Date	Recorded event
3 November 2026	Requested exact-scope EyePACS internship use and image-in-report permission. Submitted ADCIS Messidor and Messidor-2 access forms and contacted ADCIS support when personal links did not arrive.
4 November 2026	EyePACS contact directed use of its Kaggle release and requested EyePACS, Inc. and Cuadros–Bresnick (2009) credit. This did not expressly settle every proposed training, weight-sharing and image-report scope.
4 November 2026	Original EyePACS audit and grouped split completed in private storage. Earlier model two-step qualification and separate CUDA scan comparison completed on Kaggle.
4 November 2026	ADCIS technical support replied with restricted alternative access for original Messidor and Messidor-2 after the form issue. Private links and credentials are omitted here.
4 November 2026	All 12 original Messidor archives passed private size, hash and ZIP checks. The 1,200 TIFFs and 12 canonical annotation workbooks yielded a schema-valid manifest with 1,187 eligible images after 13 published duplicate exclusions.
4 November 2026	Four official Messidor-2 image parts passed byte-size and hash checks. The assembled ZIP passed CRC; all 1,748 extracted images passed full decode and byte-hash uniqueness checks. Official pairing is unresolved.
4 November 2026	Compared the block with Vim Algorithm 1 and corrected directional combination to a sum before the shared gate and projection. Earlier GPU qualification does not certify this revised source.

The official Messidor and Messidor-2 pages permit free research and educational use while prohibiting copying and redistribution. They request acknowledgment of the Messidor program ("Kindly provided by the Messidor program partners") and, for Messidor-2, the LaTIM laboratory [4, 5, 6]. Restricted access details and images are not in this dossier or the public repository.

9 Kaggle evidence and source links

1. Prepare Original EyePACS: official-source preparation, 35,126 JPEGs decoded and hashed, label reconciliation and fixed split. Produced the manifest used for subsequent qualification.
2. Full Model Qualification: checked FP32/BF16 scan parity and executed two complete optimizer steps on real EyePACS images under the pinned full-split identity. It is not an epoch or validation measurement.
3. Pinned CUDA Scan Build: built a pinned low-level selective-scan wheel in the Kaggle environment, with no pretrained weights.
4. Real EyePACS CUDA Scan Comparison: compared the in-house scan and the low-level kernel on real-image-derived inputs at intended token length. Reports kernel-level timing and numerical differences, not classifier accuracy or whole-model runtime.

These notebooks may require owner access. None is a live full training job.

10 Remaining work and limits

1. Run the exact-source and cross-cohort audit with the completed original Messidor manifest before sealed inference. The verified images and manifest alone are not external model results.
2. Requalify the current FP16 and optional CUDA full-model source, measure a complete epoch and distributed throughput, and find a zero-cost allocation that can finish a predeclared training schedule.
3. Train five-grade classification fully, selecting the checkpoint on EyePACS validation only. Preserve checkpoint and run identity.
4. Run sealed held-out EyePACS and original Messidor evaluation; generate measured metrics, plots and real example predictions. Add Messidor-2 only after its official images and pairing metadata are reconciled.
5. Compile and inspect the final *results* LaTeX report. Confirm applicable EyePACS permission before sharing image-bearing PDFs or weights.

This is research software, not a medical device or clinical diagnostic claim. Public EyePACS filenames do not independently prove patient identity. Acquisition, quality, grading-standard and population differences may affect external generalization.

References

[1] L. Zhu et al., "Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model," ICML 2024. https://arxiv.org/abs/2401.09417.

[2] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," 2023. https://arxiv.org/abs/2312.00752.

[3] J. Cuadros and G. Bresnick, "EyePACS: An Adaptable Telemedicine System for Diabetic Retinopathy Screening," *Journal of Diabetes Science and Technology*, 3(3):509–516, 2009. EyePACS, Inc. is credited for the original https://www.kaggle.com/competitions/diabetic-retinopathy-detection/data retinal image release.

[4] E. Decencière et al., "Feedback on a publicly distributed database: the Messidor database," *Image Analysis & Stereology*, 33(3):231–234, 2014. Kindly provided by the Messidor program partners. https://www.adcis.net/en/third-party/messidor/.

[5] Messidor program partners, "Messidor-2" access and acknowledgment. https://www.adcis.net/en/third-party/messidor2/.

[6] M. D. Abràmoff et al., "Automated analysis of retinal images for detection of referable diabetic retinopathy," *JAMA Ophthalmology*, 131(3):351–357, 2013. https://doi.org/10.1001/jamaophthalmol.2013.1743.

[7] Google Brain, "Messidor-2 DR Grades." https://www.kaggle.com/datasets/google-brain/messidor2-dr-grades.

Appendix A Assignment-to-source traceability

The supplied two-page internship brief is the governing requirement. Its five expected deliverables are separate evidence gates. A source file is evidence of an implementation, while a classifier result requires a completed run and inference on held-out images. The table below names the specific artifact that a reviewer can inspect for each requirement.

Brief requirement	Inspectable implementation	Current proof
Own Vim classes; no ready-made model or pretrained weights	`src/drvim/model/vim.py`, `model/scan.py`; random parameters and authored directional blocks	Local scan parity and earlier two-step GPU diagnostic; current revision needs new qualification
Own EyePACS Dataset and PyTorch DataLoader	`data/dataset.py`, `data/preprocess.py`, `data/prepare.py` and `data/split.py`	35,126 real labeled JPEGs audited; custom Dataset used in diagnostic
Training and evaluation scripts	`train.py`, `evaluate.py`, `metrics.py`, `cli.py`	Commands and local tests; full training not completed
Metrics, figures and example predictions	Result-bound evaluation and report generators	No classifier result, prediction pair or convergence graph yet
Concise compiled results report	`report.py` and a LaTeX compiler path	The present PDF documents implementation; final experimental report awaits results

What is deliberately separate

The brief does not ask for a trained clinical service, mobile application, medical device clearance, or a new dataset. The optional high-assurance loader and Messidor-2 adapter are research extensions; neither is a substitute for the required EyePACS training and original Messidor test. No published Vision Mamba score is imported as this project's score.

Review sequence

Inspect the model and scan, examine the dataset contract, reproduce local checks, confirm official source provenance, then review the exact GPU qualification and run identity. Only after a full training completion marker should the held-out evaluation and final result report be assessed. This order prevents a code handoff from being mistaken for a completed experiment.

Appendix B Tensor shapes and computational scale

The configured input has shape $B \times 3 \times 1536 \times 1536$. A convolution with 16×16 kernel and stride 16 creates a 96×96 grid. Flattening gives 9,216 patch tokens; insertion of one learned classification token gives a sequence length of 9,217. The hidden width is 768 and the directional expansion width is 1,536.

Stage	Tensor shape	Interpretation
Input image	$B \times 3 \times 1536 \times 1536$	Full field is retained by letterbox preprocessing
Patch embedding	$B \times 9216 \times 768$	One token per nonoverlapping patch
Middle class token + positions	$B \times 9217 \times 768$	Positions are learned and unique to this token sequence
Directional inner stream	$B \times 9217 \times 1536$	Forward and reverse streams have separate convolution/SSM parameters
Selective state	$B \times 1536 \times 16$ per direction	State dimension is 16; the recurrent implementation does not need to retain a full state tensor for every token at inference
Classifier output	$B \times 5$	Logits for ICDR grades 0 through 4

The depth is 24 blocks. The instantiated model has 103,760,645 trainable parameters. A dense attention matrix over the same token sequence would contain $L^2 = 84{,}953{,}089$ positions per head, before accounting for queries, keys, values or gradients. The Vim design replaces this pairwise attention operation with directional selective state updates. This count explains the architecture choice; it does not prove superior accuracy or whole-model speed for this repository.

Resolution choice

The 1536-pixel input preserves substantially more fundus detail than a small classification crop while requiring careful scan memory and runtime qualification. The assignment and paper do not mandate this resolution. The exact input geometry, patch size and token count are stored in `configs/vim_base_eye.yaml` so a later run can be reconstructed.

Where memory can grow

Patch-token activations scale with BLD per block; directional state computation also depends on expansion width and state size. Activation checkpointing recomputes selected intermediates during backward. The 103.8-million parameter count alone does not determine GPU memory: gradients, optimizer moments, decoded image batches and prefetch buffers also matter. Qualification must measure a complete model step with the chosen precision, batch and scan backend.

Appendix C Selective-state derivation and paper map

For each direction, the input-dependent projections generate B_t, C_t and a positive step size Δ_t. A learned log parameter defines a negative diagonal state matrix $A = -\exp(A_{\log})$. The implemented selective scan follows the Mamba input-drive convention:

$$\bar{A}_t = \exp(\Delta_t A), \qquad h_t = \bar{A}_t \odot h_{t-1} + \Delta_t B_t \odot x_t, \qquad y_t = \langle C_t, h_t \rangle + D \odot x_t.$$

The forward stream reads tokens left to right. The reverse stream reads a reversed copy and reverses its output back to original order. Both streams have their own depthwise causal Conv1d, state projections, step projection, $A_{\log}$ and skip vector. A shared projected gate $\mathrm{SiLU}(z)$ then modulates their *sum*, followed by output projection and residual:

$$T_\ell = T_{\ell-1} + \mathrm{DropPath}(W_o[(y_\rightarrow + y_\leftarrow) \odot \mathrm{SiLU}(z)]) \,.$$

The directional sum matches the paper's Algorithm 1. A code review on 4 November corrected an earlier factor of one-half in this expression. The existing two-step GPU diagnostic was run before this correction and must not be presented as qualification of the present source revision.

Reference, parallel and optional kernel paths

The reference recurrence is intentionally direct and useful for numerical comparison. The in-house parallel path composes affine transitions in bounded chunks and implements backward propagation. An optional pinned CUDA selective-scan dependency is a low-level efficiency path, never a ready-made Vim model or a source of pretrained weights. Its benchmark is scan-only; the corrected whole classifier has not been profiled with it.

Paper boundary

The cited paper studies ImageNet and downstream dense prediction. This project changes the task to five-grade retinal classification, initializes weights randomly, uses 1536-pixel letterboxed inputs, and defines its own training schedule. These task decisions and this implementation's timings must not be confused with the paper's reported benchmark results [1, 2].

Appendix D Official cohort provenance

The three data sources have distinct roles and label semantics. The original EyePACS Kaggle training release supplies five-grade labels and is the only training source. Original Messidor is reserved for external evaluation. Messidor-2 is optional and partly overlaps original Messidor.

Source	Verified source counts	Research use in this project
EyePACS Kaggle	35,126 labeled JPEGs; grade counts 25,810 / 2,443 / 5,292 / 873 / 708	Train, validation and sealed held-out test under ICDR grades 0–4
Original Messidor / ADCIS	1,200 TIFFs, 12 canonical workbooks; 1,187 eligible after 13 published duplicate exclusions	External test under R0–R3; cross-source referable proxy only
Messidor-2 / ADCIS	1,748 decoded images: 1,058 PNGs and 690 JPGs; independent Google Brain grades cover 1,744 gradable images	Optional five-grade analysis after official pairing metadata is verified

Every original Messidor TIFF was decoded and SHA-256 hashed; four published grade corrections were applied. Eligible R0/R1/R2/R3 counts are 547/149/240/251. Its manifest passed schema validation. The included EyePACS and original Messidor image-hash sets have zero exact overlap. That comparison does not detect recoded near duplicates or prove independent patient identity.

The Messidor-2 ZIP passed CRC and all 1,748 extracted images passed a full decode and byte-hash uniqueness audit. Its 1,058 PNG filename stems match original Messidor TIFF stems; its 690 JPG stems do not. ADCIS describes those PNGs as the original paired subset and the JPGs as the extension. Reporting Messidor and whole Messidor-2 as independent replications would therefore overstate the evidence. The official pairing spreadsheet was not present in the delivered image ZIP, so no Messidor-2 result is claimed [4, 5, 7].

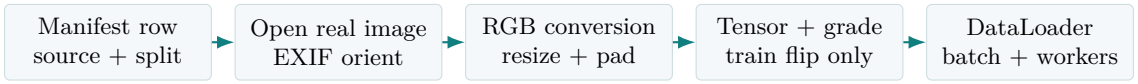
Observed native image dimensions

Native dimensions	Original Messidor	Messidor-2	Purpose
1440×960	588	528	Prevent silent geometry assumptions
2240×1488	400	616	Verify decode and resize inputs
2304×1536	212	604	Preserve the field at every size

These counts come from the private image manifest and full decode audit, not classifier inference. The normalizer retains each rectangular source field in a square canvas; it does not claim identical camera or color distributions across cohorts.

Appendix E Custom Dataset and image contract

The active `RetinaDataset` is the project's own map-style PyTorch `Dataset`. PyTorch's `DataLoader` supplies workers and batching, as the assignment requests. No ready-made retinal dataset loader or synthetic example cohort is used. A sample is selected from a validated manifest row with source, image ID, path, grade, split, inclusion flag, dimensions and SHA-256 identity.



Aspect-preserving resize uses scale $s = \min(1536/w, 1536/h)$; the resized image is centered in a black 1536×1536 canvas. This retains the whole source field instead of cutting off peripheral retinal content. Horizontal reflection is a training-only augmentation. Validation and test use a deterministic transform. The model receives RGB channel-first tensors and a five-grade integer label for EyePACS.

Separate high-assurance candidate

The additional module implements stricter per-read byte hashing, explicit format and size limits, ICC-to-sRGB conversion and recorded untagged-color assumptions, deterministic augmentation keys, and metadata requirements. It is not connected to the primary training run. The public EyePACS release lacks confirmed patient, site, device, adjudication and rights metadata, so the candidate fails closed rather than inventing them. This distinction is essential when evaluating possible medical-grade or licensable use: a strict interface cannot create missing clinical proof.

Sample-level failure decisions

Condition	Required behavior
Unlabeled or repeated ID	Preparation fails; no target grade is invented
Unreadable, invalid-size or non-color image	Decode/manifest check fails before training
Image bytes change	Hash audit fails; the source is no longer frozen
Excluded or ungradable record	Explicit exclusion remains in manifest and is omitted from the learning or result denominator
Unknown patient/site/color metadata	No guessed value; the optional strict candidate rejects absent attestations

Appendix F Split design and leakage boundaries

The labeled EyePACS release was audited as 35,126 images. Filename-derived left/right pairing IDs were grouped before assignment, and grades were stratified. The frozen split contains 24,590 training, 5,268 validation and 5,268 held-out test images. All five grades occur in each partition. No filename-pair group crosses partitions. The split manifest SHA-256 is `f933f1bcff8f99688fd8a44984acaa4ce0c929d899be9bddcabdfc54ea7c2117`.

Actual EyePACS image counts by grade

Partition	G0	G1	G2	G3	G4	Total
Training	18,060	1,713	3,693	622	502	24,590
Validation	3,872	365	806	130	95	5,268
Held-out test	3,878	365	793	121	111	5,268

Grade 0 dominates each partition, while grades 3 and 4 are rare. Macro metrics and class-level confusion counts will therefore matter more than an isolated accuracy number. These are label counts, not measurements of classifier sensitivity or agreement.

Partition	Images	Permitted role
Training	24,590	Gradient updates and training-only augmentation
Validation	5,268	Checkpoint selection and declared training diagnostics
Held-out EyePACS	5,268	One sealed internal test after full training
Original Messidor	1,187 eligible	External test only; never fine-tune or choose a threshold on it

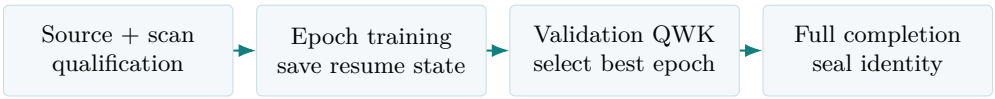
The filename token is a grouping proxy, not an independently verified patient identifier. The public release does not supply complete patient, site or device linkage. Exact byte-hash overlap between the included EyePACS and original Messidor manifests is zero; perceptual near duplicates, related subjects and acquisition correlations remain separate questions. The optional Messidor-2 cohort overlaps original Messidor by design, so those two cannot be pooled as independent external tests.

Sealed-test rule

The training loop may inspect training loss and validation QWK. It does not consume held-out EyePACS or Messidor labels. The evaluation command requires a training-complete marker, a selected validation checkpoint, manifest hashes and an audit artifact. It refuses to overwrite existing test results. This design makes test reuse visible and reduces accidental selection on held-out outcomes; it does not replace an independent prospective validation study.

Appendix G Training lifecycle and checkpoint identity

The configured candidate uses AdamW, cross-entropy, 10^{-4} initial learning rate, 0.05 weight decay, five warmup epochs and cosine decay. Per-GPU batch is two, with four gradient-accumulation steps. For G participating GPUs, the intended effective batch is $2 \times 4 \times G$ real images per optimizer update, subject to end-of-epoch handling. FP16 automatic mixed precision, gradient clipping at norm 1.0, activation checkpointing, distributed sampling and a maximum drop-path rate of 0.1 are configured. The declared ceiling is 300 epochs; it is a project choice, not a requirement of the brief or the Vim paper.



The checkpoint stores model, optimizer, scheduler, scaler and random number generator state with configuration, manifest and source identities. An epoch-boundary pause writes resumable state without marking a full run complete. Resume requires the same source, data split, configuration, qualification and distributed world size. The best checkpoint is selected by validation QWK alone.

Checkpoint integrity fields

Recorded state	Why it is needed
Model and optimizer	Resume parameters and moments from the same epoch
Scheduler and gradient scaler	Preserve learning-rate and precision state
Random generators	Make stochastic order and augmentation auditable
Source, manifest and config hashes	Reject a checkpoint paired with changed code, split, images or hyperparameters
Qualification identity	Prevent a diagnostic from another scan backend or model revision from being reused

Evidence boundary

An earlier BF16/parallel version completed two real-image optimizer steps on one T4. The current source has FP16 and optional CUDA changes, and its directional combination was corrected to the paper's sum. No complete epoch, resume cycle, distributed throughput, trained checkpoint or validation score has been demonstrated for this exact revision. Its training lifecycle is implemented but not yet experimentally qualified.

Appendix H Evaluation definitions and outputs

For comparable five-grade ICDR labels, the per-image record contains source ID, true grade, predicted grade and five probabilities. The confusion matrix has $C_{ij} = \#\{n : y_n = i, \hat{y}_n = j\}$. For each class, precision is $TP/(TP + FP)$, recall is $TP/(TP + FN)$ and F1 is $2PR/(P + R)$ under the code's explicit zero-denominator policy. Macro values average the five class values; accuracy is $\sum_i C_{ii} / \sum_{ij} C_{ij}$. QWK compares observed and expected grade-pair frequencies with quadratic distance weights $w_{ij} = (i - j)^2/(K - 1)^2$. Undefined QWK is rejected.

Evaluation cohort	Permitted result interpretation
Held-out EyePACS	Five-grade accuracy, per-class and macro precision/recall/F1, QWK, confusion matrix and referable proxy
Original Messidor	Its R0–R3 grades are not ICDR 0–4. Report the predeclared binary proxy: true R2/R3 against predicted ICDR 2/3/4; five-grade QWK against these labels would be invalid
Optional Messidor-2	Five-grade metrics only after official images, pairing metadata and independent adjudicated grades reconcile; disclose overlap with original Messidor

The evaluation script verifies the selected checkpoint and source hashes, runs inference, saves per-image rows, then computes metric JSON and plots. The report generator recalculates results from saved rows before placing confusion matrices, curves and real source image–prediction pairs in a LaTeX report. This chain is implemented but has not yet produced a classifier result. No number in this dossier is a DR accuracy, F1 or QWK measurement for the model.

Required final artifact chain

A completed run should create a validation-selected checkpoint and training-complete record. The sealed evaluation then writes per-image predictions, cohort metric JSON and confusion matrices. The report step must compile the results TeX, place real image–prediction pairs only when permitted, and tie displayed numbers to prediction rows. Training and validation curves come from the epoch log; the external test cannot contribute to those curves or checkpoint choice. Cohort denominators must reflect explicit exclusions, including the 13 published original Messidor duplicate removals.

Appendix I Measured diagnostics and feasibility

The completed Kaggle preparation job checked all 35,126 original EyePACS images and produced the frozen split. A separate full-manifest diagnostic ran an earlier model source for exactly two eight-image optimizer updates on one T4 at 1536 pixels. Those updates took 195.71 and 197.24 seconds. These are measured step times, not an epoch duration or a trained model.

Observed check	Value	Scope
EyePACS preparation	35,126 audited images	Source/label audit and grouped split only
Earlier full-model steps	195.71 s; 197.24 s	Two real-image updates, one T4, earlier BF16/parallel revision
Scan-only in-house path	0.33374 s	One measured 9,217-token scan benchmark, not a classifier step
Scan-only pinned CUDA path	0.02563 s	Same low-level benchmark; maximum forward absolute difference 1.31×10^{-6}
Repository checks	Ruff pass; 12 tests pass	Source checks, not clinical or accuracy qualification

At the earlier step rate, $24{,}590/8 \approx 3{,}074$ optimizer updates per epoch would imply roughly 168 hours for one epoch before validation, I/O, checkpoints and any distributed speedup. This is a conditional extrapolation, not a timed epoch. The optional low-level CUDA scan showed about a 13-fold improvement for its measured kernel path, but this cannot be multiplied into a whole-model training estimate. The current source needs a full-model GPU qualification and a measured epoch. No paid compute has been used; the available zero-cost Kaggle quota has not been shown sufficient for the declared schedule.

Appendix J Failure controls, credits and handoff

The code rejects missing or repeated labels, unknown grade schemes, invalid images, duplicate included image hashes, altered manifests, source/checkpoint mismatches and test evaluation before training completion. It records exclusions rather than silently dropping rows. Its high-assurance candidate adds byte, format, color and metadata gates but is not an active trainer dependency.

Risk	Implemented or required response
Incorrect Messidor labels	Apply the four official errata; exclude one member of each of 13 published Base33 duplicate pairs
Patient/eye leakage	Keep filename-paired EyePACS eyes together; acknowledge that the token is not verified patient identity
Cross-source label mismatch	Use the declared binary referable proxy for original Messidor; reserve five-grade QWK for comparable labels
Incomplete or changed run	Pin source/config/manifest/checkpoint hashes; require completion marker and refuse result overwrite
Dataset rights	Keep images, credentials and restricted ADCIS access outside Git; obtain exact EyePACS scope clarification before sharing image-bearing results or weights

Project and source links

GitHub source repository contains the code and this TeX/PDF. The owner-controlled Kaggle notebooks linked on page 7 separately document EyePACS preparation, two-step qualification, kernel build and scan comparison. EyePACS, Inc., Cuadros–Bresnick (2009), the Vision Mamba authors, the Messidor program, LaTIM, ADCIS and the independent Google Brain Messidor-2 graders are credited in the reference list and README.

Submission boundary

This compiled implementation dossier and the first three code deliverables are ready for a code-focused review. The assignment's full experimental submission still requires completed training, measured EyePACS and original Messidor results, real prediction examples and a separate concise results report. Those outputs are not represented as finished here.